Análise Empírica de Desempenho e Complexidade de Algoritmos de Ordenação com Java

Bruno Benitez Forgiarini[1], **Maria Eduarda Maia Almeida**[1]

[1]Universidade Federal do Pampa (Unipampa)

brunoforgiarini.aluno@unipampa.edu.br

mariaema2.aluno@unipampa.edu.br

Abstract. *This report presents a comprehensive empirical study on the computational efficiency of ten sorting algorithms (BubbleSort, SelectionSort, InsertionSort, ShellSort, MergeSort, QuickSort, HeapSort, CountingSort, RadixSort, and BucketSort) implemented in Java. The experiment evaluated the impact of asymptotic complexities $O(n^2)$, $O(n \log n)$, and $O(n)$, initial data distributions (Random, Sorted, Reversed, and Repeated), and the Object-Oriented overhead across different data types (`Integer`, `String`, and a custom `Pessoa` class). The results experimentally demonstrate the drastic scaling divergence between complexity classes and reveal the cost of abstractions in the JVM.*

Resumo. *Este relatório apresenta um estudo empírico abrangente sobre a eficiência computacional de dez algoritmos de ordenação (BubbleSort, SelectionSort, InsertionSort, ShellSort, MergeSort, QuickSort, HeapSort, CountingSort, RadixSort e BucketSort) implementados na linguagem Java. O experimento avaliou o impacto das classes de complexidade assintótica $O(n^2)$, $O(n \log n)$ e $O(n)$, das distribuições iniciais dos dados (Aleatório, Ordenado, Invertido e Repetidos) e do impacto do tipo de dado e da representação utilizada no desempenho através de tipos distintos (`Integer`, `String` e a classe genérica `Pessoa`). Os resultados evidenciam experimentalmente a drástica divergência de escala entre as classes de complexidade e revelam o custo das abstrações na JVM.*

1. Introdução

Este relatório técnico apresenta o desenvolvimento, a verificação e a análise experimental de desempenho de dez algoritmos de ordenação, desenvolvidos como parte do Trabalho Prático 1 (TP1) da disciplina de Algoritmos e Classificação de Dados do curso de Engenharia de Computação da UNIPAMPA. O escopo principal deste estudo é analisar os métodos de ordenação de dados para o propósito de avaliação, com respeito ao tempo de execução e ao comportamento de desempenho na prática, a partir da perspectiva do desenvolvedor.

Para conduzir essa investigação, foram implementados em Java sete algoritmos baseados em comparação (Bubble Sort, Insertion Sort, Selection Sort, Shell Sort, Heap Sort, Merge Sort e Quick Sort) e três algoritmos de tempo linear (Counting Sort, Radix Sort e Bucket Sort). A avaliação experimental submeteu esses algoritmos a diferentes cenários de estresse, variando o tamanho das coleções (de 100 até 1.000.000 de elementos), a disposição inicial dos dados (ordem aleatória, crescente, invertida e com várias ocorrências

repetidas) e os tipos de dados processados (tipos primitivos como `int[]` e objetos como `Integer`, `String` e uma classe customizada `Pessoa`).

O objetivo central desta análise vai além da simples identificação dos menores tempos absolutos de execução. Busca-se interpretar os dados coletados (totalizando 4.440 execuções válidas) para identificar as características das entradas que mais influenciam a eficiência de cada método e relacionar diretamente os resultados observados na prática com a análise teórica de complexidade assintótica estudada ao longo do componente curricular.

2. Fundamentação Teórica e Algoritmos Analisados

Modelo RAM e Notação Assintótica

A análise da complexidade dos algoritmos baseia-se no modelo *Random Access Machine* (RAM), que assume que instruções básicas (como operações aritméticas, atribuições e comparações) são executadas em tempo constante. A notação assintótica Big-O é utilizada para expressar o limite superior do tempo de execução em função do tamanho da entrada n. Para algoritmos de ordenação estritamente baseados em comparações, a teoria estabelece um limite inferior de $\Omega(n \log n)$ para o pior caso. Um resumo das complexidades teóricas dos algoritmos avaliados está expresso na Tabela 1.

Tabela 1. Complexidade Teórica dos Algoritmos de Ordenação Estudados

Algoritmo	Melhor Caso	Caso Médio	Pior Caso	Espaço Auxiliar	Estável?
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Sim
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	Não
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Sim
Shell Sort	$O(n \log n)$	$O(n^{1.5})$	$O(n^2)$	$O(1)$	Não
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Sim
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	Não
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	Não
Counting Sort	$O(n + k)$	$O(n + k)$	$O(n + k)$	$O(n + k)$	Sim
Radix Sort	$O(d(n + k))$	$O(d(n + k))$	$O(d(n + k))$	$O(n + k)$	Sim
Bucket Sort	$O(n + k)$	$O(n + k)$	$O(n^2)$	$O(n + k)$	Sim

Algoritmos Comparativos $O(n^2)$

Os algoritmos com complexidade de pior caso $O(n^2)$ implementam lógicas simples e apresentam bom desempenho apenas para coleções pequenas ou já quase ordenadas:

- **Bubble Sort:** Percorre repetidamente o vetor, comparando elementos adjacentes e trocando suas posições quando estão fora de ordem. Em cada passagem, os maiores elementos da região percorrida tendem a deslocar-se para o final do vetor. O processo é repetido até que os elementos estejam ordenados. Sua implementação possui complexidade de tempo $O(n)$ no pior caso e utiliza espaço auxiliar $O(1)$.
- **Selection Sort:** Divide conceitualmente o vetor em uma região ordenada e outra não ordenada. A cada iteração, procura o menor elemento da região não ordenada e o troca com o primeiro elemento dessa região. Ao final de cada etapa, mais uma posição passa a integrar a região ordenada. O algoritmo realiza $\theta(n)$ comparações

mesmo quando a entrada já está ordenada, utiliza espaço auxiliar $O(1)$ e realiza poucas trocas em comparação com outros métodos simples.
- **Insertion Sort:** Remove sequencialmente cada item da parte não ordenada e o insere na posição correta dentro da sublista previamente ordenada.

Algoritmos Comparativos $O(n \log n)$

- **Bubble Sort:** Compara elementos adjacentes e troca aqueles que estão fora de ordem, repetindo o processo até ordenar o vetor.
- **Selection Sort:** Busca o menor elemento da região não ordenada e o troca com o primeiro elemento dessa região. Realiza $O(n^2)$ comparações.
- **Insertion Sort:** Insere cada elemento na posição correta de uma região previamente ordenada. Seu desempenho melhora em vetores já ordenados ou quase ordenados.

2.1. Algoritmos Comparativos de Ordenação Avançados

Esses algoritmos utilizam estratégias que reduzem o custo da ordenação ou organizam os elementos por intervalos e estruturas auxiliares.

- **Shell Sort:** Extensão do *Insertion Sort* que utiliza intervalos decrescentes. Nesta implementação, adota a sequência de Knuth ($h = 3h + 1$). Sua complexidade depende da sequência utilizada.
- **Merge Sort:** Divide recursivamente o vetor e intercala as partes ordenadas. Apresenta tempo $O(n \log n)$ e espaço auxiliar $O(n)$.
- **Quick Sort com Particionamento de 3 Vias:** Utiliza o elemento central como pivô e separa valores menores, iguais e maiores que ele. Ordena recursivamente o menor intervalo e processa o maior por laço, reduzindo a profundidade da pilha. Seu pior caso continua sendo $O(n^2)$.
- **Heap Sort:** Utiliza um *max-heap* para selecionar repetidamente o maior elemento. Garante tempo $O(n \log n)$ e espaço auxiliar $O(1)$ na implementação *in-place*.

2.2. Algoritmos Não Comparativos

Esses algoritmos exploram características dos valores de entrada, como amplitude numérica e quantidade de dígitos.

- **Counting Sort:** Conta as ocorrências dos valores para determinar suas posições. Apresenta tempo e espaço auxiliar $O(n + k)$, em que k é a amplitude numérica. A implementação também permite números negativos por meio de um deslocamento.
- **Radix Sort:** Ordena os números dígito a dígito, utilizando uma ordenação estável por contagem. Sua complexidade é $O(dn)$, considerando base decimal fixa, em que d representa a quantidade de dígitos. A implementação aceita inteiros não negativos e pressupõe vetor não vazio.
- **Bucket Sort:** Distribui os elementos em n baldes, ordena cada balde com `Collections.sort()` e concatena os resultados. Seu desempenho depende da distribuição dos valores e do custo da ordenação interna.

Considerações de viabilidade: O Counting Sort utiliza um vetor de contagem cujo tamanho corresponde à amplitude dos valores de entrada, calculada como a diferença entre o maior e o menor elemento, acrescida de uma unidade. Assim, sua complexidade temporal é $(O(n + k))$, em que (n) representa o número de elementos e (k) corresponde à amplitude

considerada, enquanto o consumo de memória auxiliar é $(O(n+k))$ nesta implementação, devido ao vetor de saída e ao vetor de contagem.

O Radix Sort implementado processa os números inteiros em base decimal, ordenando os elementos de acordo com cada dígito por meio de uma versão estável do Counting Sort. Para cada posição decimal, utiliza um vetor auxiliar de tamanho (n) e um vetor de contagem com dez posições. Sua complexidade temporal é $(O(d(n+10)))$, equivalente a $(O(dn))$ para a base decimal fixa, em que (d) representa a quantidade de posições decimais processadas. O consumo de memória auxiliar é $(O(n+10))$ por passagem. Diferentemente do Counting Sort implementado, o Radix Sort não aloca um vetor de contagem proporcional à amplitude numérica total dos dados.

3. Metodologia Experimental

A avaliação experimental foi planejada e executada de forma automatizada para garantir a reprodutibilidade dos resultados, o rigor estatístico e o isolamento de interferências do sistema operacional.

3.1. Pipeline de Desenvolvimento e Verificação

O fluxo de desenvolvimento foi dividido nas seguintes etapas sequenciais:

1. **Implementação dos Algoritmos:** Construção das classes dos dez algoritmos de ordenação na linguagem Java, utilizando a interface `Comparable<T>` para genéricos e arranjos `int[]` para algoritmos lineares.
2. **Testes de Corretude (`TestesCorretude`):** Desenvolvimento de testes unitários para validar a exatidão em vetores pequenos (10 a 15 elementos) sob casos especiais de entrada.
3. **Gerador de Entradas (`GeradorDeEntradas`):** Módulo responsável por instanciar os arranjos. Para garantir a reprodutibilidade, utilizou-se `java.util.Random` com semente fixa (`seed = 42`), estruturando os cenários da seguinte forma:
    - **Configurações de Entrada:** O cenário *Aleatório* sorteia chaves no intervalo $[0, 10n)$; o *Ordenado* preenche o vetor de forma crescente ($i = 0 \ldots n-1$); o *Invertido* dispõe os valores em ordem decrescente ($n-1-i$); e o *Repetidos* sorteia números estritamente no intervalo $[0, 10)$, garantindo alta taxa de duplicatas.
    - **Adaptação por Tipo de Dado:** Nos vetores de `Integer` e `int[]`, os valores gerados são atribuídos diretamente. Para `String`, os números são formatados com máscara fixa de 8 dígitos (`"STR_%08d"`), assegurando que a ordenação lexicográfica reflita exatamente a grandeza numérica. Para a classe `Pessoa`, a chave define o atributo comparável (`id`), enquanto a "carga útil" do objeto (`nome`, `idade` e `salario`) é preenchida deterministicamente a partir do índice.
4. **Motor de Benchmarking (`BenchmarkRunner`):** Executável principal encarregado de orquestrar os aquecimentos da JVM, clonar os vetores a cada iteração, cronometrar a ordenação via `System.nanoTime()` e exportar os dados brutos.

3.2. Parâmetros do Experimento e Tipos de Dados

O experimento avaliou o desempenho combinando os seguintes fatores:

- **Tamanhos de Entrada (N):** Vetores variando de 100 até 1.000.000 elementos. Devido ao tempo proibitivo (onde algoritmos quadráticos superaram 35 segundos para $N = 100.000$), *BubbleSort*, *SelectionSort* e *InsertionSort* foram **excluídos do cenário** $N = 1.000.000$.
- **Cenários de Distribuição Inicial:** *Aleatório*, *Ordenado*, *Invertido* (pior caso) e *Repetidos* (alta densidade de chaves duplicadas).
- **Tipos de Dados:** O experimento cobriu arranjos primitivos (`int[]`), encapsuladores genéricos puros (`Integer`), cadeias léxicas (`String` no formato `"STR_%08d"` com custo interno $O(L)$) e objetos estruturados na *Heap* (`Pessoa`).

3.3. Sanitização, Tratamento de Dados e Análise Estatística

Cada cenário experimental foi submetido a 3 rodadas iniciais de aquecimento da JVM (*warm-up*), descartadas da contagem, seguidas por 10 repetições oficiais.

Os dados brutos coletados pelo `BenchmarkRunner` foram exportados em milissegundos (ms). O arquivo de resultados passou por uma etapa automatizada de tratamento através de um script em Python, utilizando as bibliotecas *Pandas* e *Matplotlib* para a geração do resumo estatístico e dos gráficos, respectivamente. A partir das 10 medições de cada configuração, calculou-se o resumo estatístico completo contendo: Média (μ), Desvio Padrão (σ), Mediana (Q_2), valores de Mínimo e Máximo (Amplitude) e Quartis (Q_1, Q_3). Na Tabela 2, apresentamos de forma consolidada as métricas centrais (μ, σ e Q_2) para os maiores volumes de dados.

3.4. Ambiente de Hardware e Software

O benchmark foi executado em ambiente dedicado (AMD Ryzen™ 5 5600 3.5 GHz, 16 GiB RAM DDR4, SSD 1.7 TB) rodando Ubuntu 24.04.5 LTS (Kernel Linux 7.0.0). Os testes operaram na OpenJDK 17 com o sistema em estado de repouso (*idle*) absoluto, sem aplicativos secundários em segundo plano durante aproximadamente 3 horas de execução.

4. Resultados e Discussão

A avaliação gerou um vasto conjunto de métricas empíricas. Os resultados foram agrupados considerando escalas de tamanho e tipos de dados, utilizando as médias da Tabela 2 como referência exata para as análises gráficas.

Tabela 2. Resumo Estatístico Consolidado: Cenário Aleatório ($N = 100.000$ e $N = 1.000.000$)

Algoritmo	Tipo	Tempo Médio (ms) (μ)	Desvio Padrão (σ)	Mediana (Q2)
Tamanho $N = 100.000$				
BubbleSort	`Integer`	36289,66	57,92	36271,01
HeapSort	`Integer`	24,41	0,07	24,42
InsertionSort	`Integer`	13573,81	13,25	13571,89
MergeSort	`Integer`	17,36	0,13	17,33
QuickSort	`Integer`	18,67	0,66	18,38
SelectionSort	`Integer`	23870,97	23,84	23867,88
ShellSort	`Integer`	29,90	0,22	29,81
BubbleSort	`Pessoa`	38745,55	53,86	38730,26
HeapSort	`Pessoa`	26,38	0,07	26,37
InsertionSort	`Pessoa`	14724,36	24,46	14718,51
MergeSort	`Pessoa`	18,71	0,03	18,72
QuickSort	`Pessoa`	20,97	0,60	20,79
SelectionSort	`Pessoa`	26198,14	132,80	26221,40
ShellSort	`Pessoa`	31,57	0,10	31,56
BubbleSort	`String`	89069,35	128,19	89108,85
HeapSort	`String`	49,95	0,22	49,90
InsertionSort	`String`	27980,66	12,51	27981,57
MergeSort	`String`	30,63	0,04	30,64
QuickSort	`String`	36,15	0,89	35,86
SelectionSort	`String`	51770,04	1351,77	52178,71
ShellSort	`String`	65,09	1,87	64,43
BucketSort	`int[]`	5,47	0,63	5,27
CountingSort	`int[]`	3,45	0,08	3,45
RadixSort	`int[]`	2,45	0,03	2,47
Tamanho $N = 1.000.000$				
HeapSort	`Integer`	396,72	1,38	396,59
MergeSort	`Integer`	281,11	27,56	270,35
QuickSort	`Integer`	320,36	3,34	319,40
ShellSort	`Integer`	557,80	1,24	557,83
HeapSort	`Pessoa`	632,57	8,56	632,16
MergeSort	`Pessoa`	409,34	4,95	407,77
QuickSort	`Pessoa`	498,96	39,68	501,33
ShellSort	`Pessoa`	1052,10	6,24	1051,37
HeapSort	`String`	945,29	4,26	944,53
MergeSort	`String`	501,13	2,40	500,52
QuickSort	`String`	576,17	4,34	575,08
ShellSort	`String`	1694,27	12,18	1693,26
BucketSort	`int[]`	178,93	19,10	175,36
CountingSort	`int[]`	49,77	0,74	49,68
RadixSort	`int[]`	31,76	5,59	28,93

4.1. Comparação Geral de Classes de Complexidade

Para estabelecer uma linha de base, avaliou-se o caso médio (vetores com disposição aleatória) dividindo a análise entre os algoritmos comparativos e os de tempo linear.

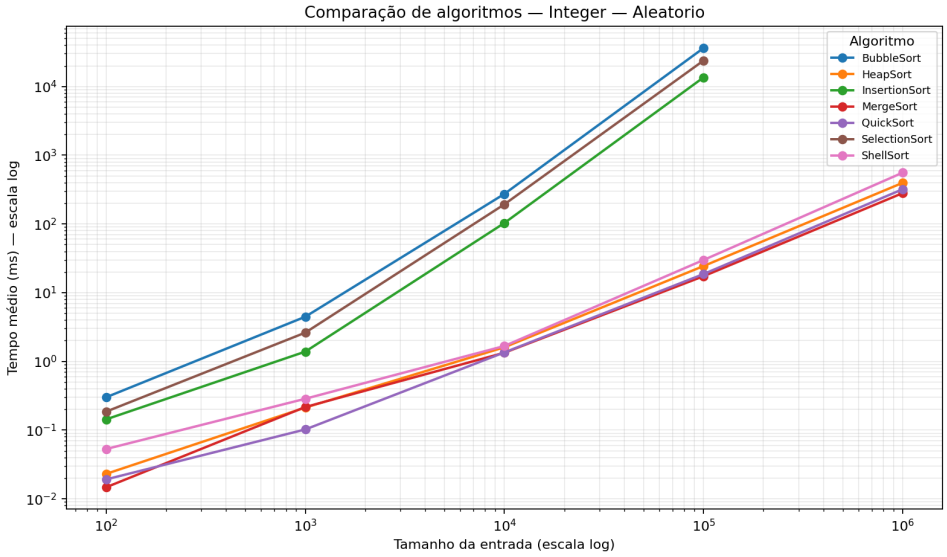


Figura 1. Comparação geral em arranjos aleatórios para algoritmos baseados em comparação (`Integer`).

Conforme suportado pela Tabela 2, a Figura 1 apresenta uma divisão clara entre as complexidades. Para $N = 100.000$, algoritmos $O(n^2)$ como *Bubble Sort* sofrem uma escalada abrupta de tempo (chegando a brutais 36.289 ms). Em contrapartida, os $O(n \log n)$ mantêm uma curva de crescimento suave (com o *Merge Sort* cravando $17{,}36$ ms).

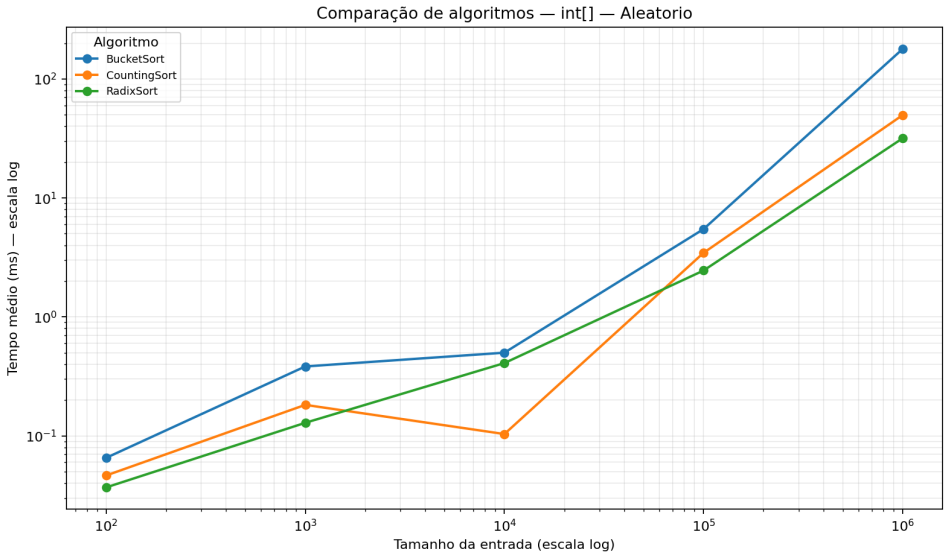


Figura 2. Comparação geral em arranjos aleatórios para algoritmos de tempo linear (`int[]`).

A Figura 2 ilustra a execução sobre `int[]`, onde operam os algoritmos não-comparativos. Evidencia-se a eficácia dessas abordagens espaciais, que alcançam tempos substancialmente reduzidos (o *RadixSort* registrou $2{,}45$ ms para $N = 100.000$ e apenas $31{,}76$ ms para $N = 1.000.000$, conforme a Tabela 2). Nota-se uma inflexão atípica na

curva do *CountingSort* entre 10^3 e 10^4 elementos, fenômeno decorrente da compilação em camadas da JVM que será detalhado na análise de perfil individual (Seção 4.5).

4.2. Sensibilidade ao Estado Inicial e Casos Limites

O grau de ordenação afeta drasticamente o desempenho empírico.

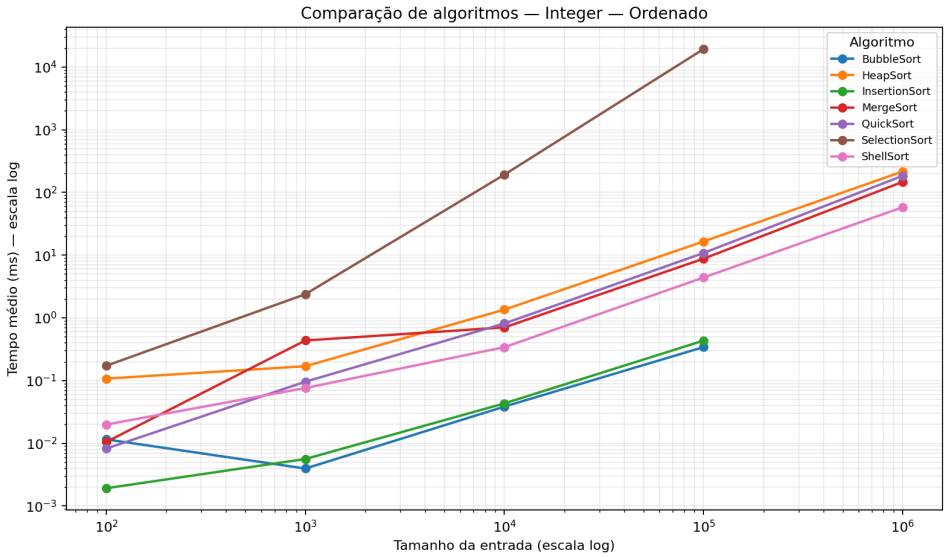


Figura 3. Impacto do estado inicial: Melhor Caso (vetor previamente Ordenado).

Na Figura 3, destaca-se a forte adaptabilidade do *InsertionSort* e do *BubbleSort* em vetores já ordenados. Este desempenho excepcional do *BubbleSort* deve-se à sua implementação otimizada: a utilização de uma *flag* booleana de controle (`troca = false`). Como não ocorrem trocas na primeira passagem, o laço é interrompido precocemente, garantindo a ambos os algoritmos um custo linear $O(n)$ e máxima eficiência. Em contraste, o *SelectionSort* não é adaptativo: ignora a ordenação prévia, mantém o volume máximo de comparações ($O(n^2)$) e apresenta a maior latência do gráfico.

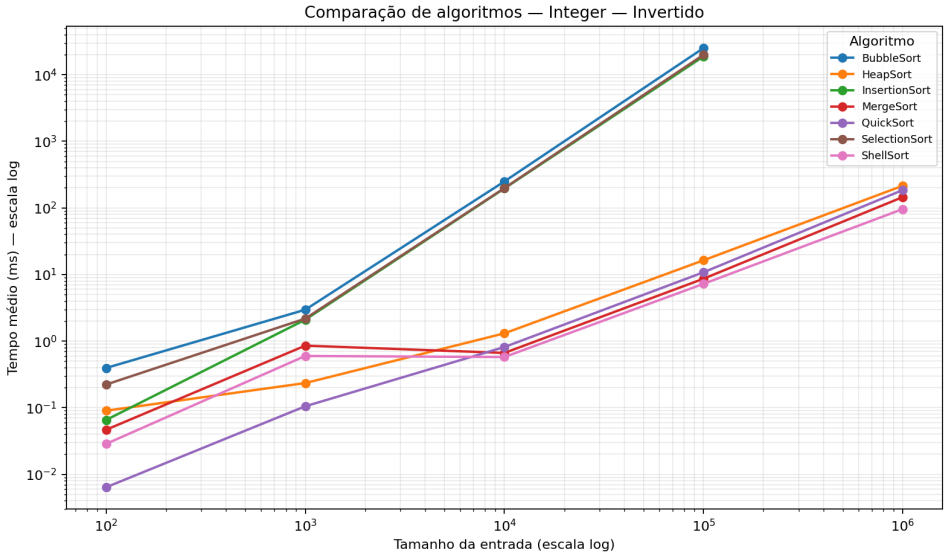


Figura 4. Impacto do estado inicial: Pior Caso (vetor Invertido).

A Figura 4 ilustra o impacto do pior caso (vetor em ordem reversa). Neste cenário, a otimização do *BubbleSort* torna-se inútil, e tanto ele quanto o *InsertionSort* degradam drasticamente para $O(n^2)$, perdendo a sua vantagem ao realizarem o máximo de movimentações de memória. Em contrapartida, os algoritmos avançados (*MergeSort*, *HeapSort*, *QuickSort* e *ShellSort*) demonstram resiliência, mantendo as suas curvas estáveis próximas a $O(n \log n)$ e garantindo alto desempenho mesmo na pior disposição de dados.

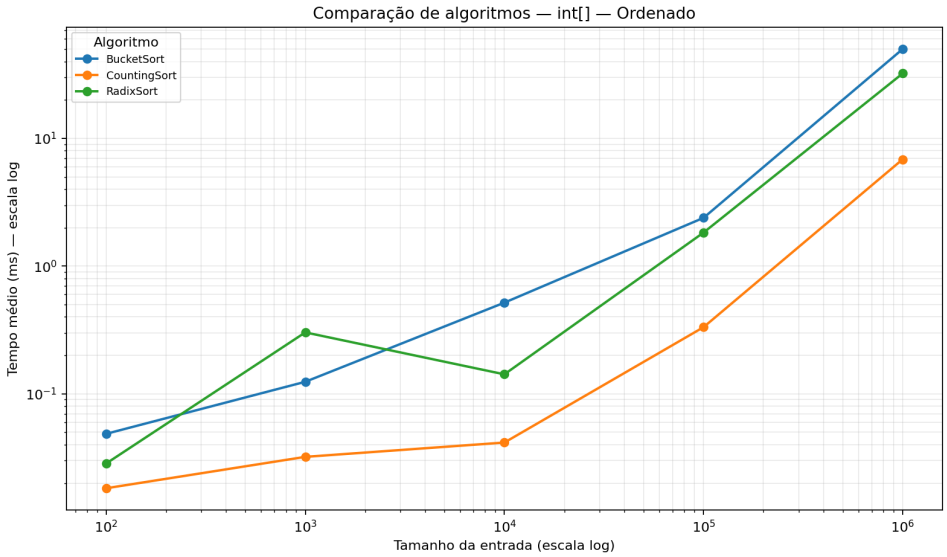


Figura 5. Impacto do estado inicial: Melhor Caso (vetor Ordenado).

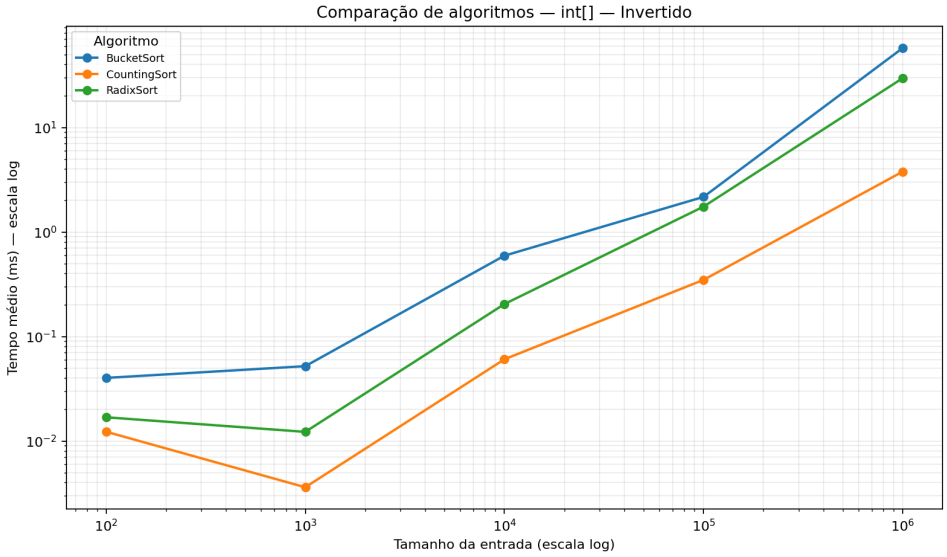


Figura 6. Impacto do estado inicial: Pior Caso (vetor Invertido).

As Figuras 5 e 6 ilustram o desempenho dos algoritmos de tempo linear (*BucketSort*, *CountingSort* e *RadixSort*) operando sobre tipos primitivos (`int[]`) nos cenários limite de entrada: Ordenado (Melhor Caso) e Invertido (Pior Caso).

Em ambos os cenários, observa-se que a natureza não-comparativa desses algoritmos os torna praticamente imunes à ordenação prévia do vetor. As curvas de crescimento mantêm a mesma trajetória linear, comprovando que o esforço computacional baseia-se estritamente na dimensão da entrada (N) e na dispersão dos valores (tamanho do balde ou contagem de frequências), e não na posição relativa pré-existente dos elementos. Destaca-se o *CountingSort*, que sustentou a menor latência em todas as ordens de grandeza em ambos os gráficos, devido à sua simplicidade de indexação direta na memória.

4.3. Impacto do Tipo de Dado e Cenários de Entrada

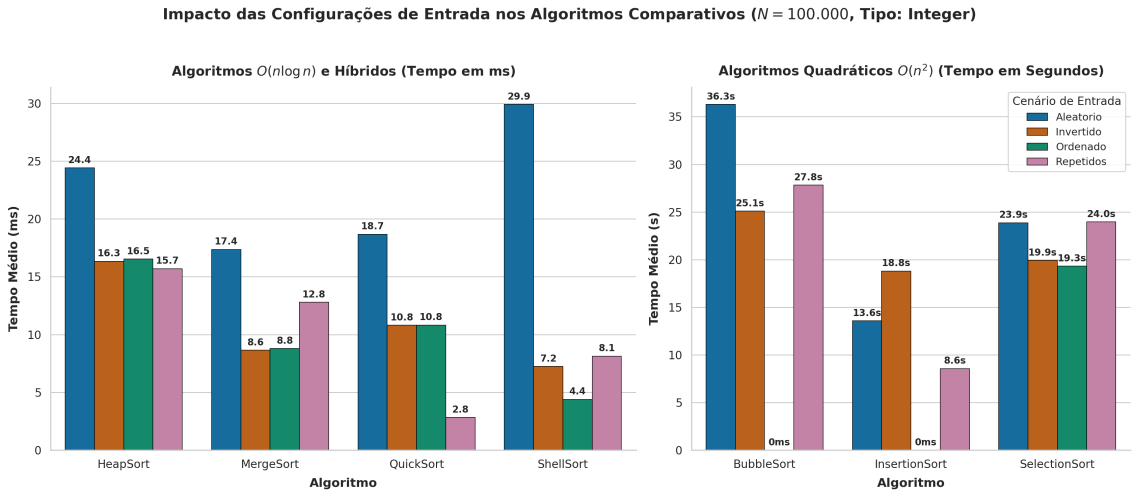


Figura 7. Desempenho nos casos Aleatório, Invertido, Ordenado e Repetido com ordenação do tipo `Integer`.

A Figura 7 consolida o comportamento dos algoritmos sob quatro configurações iniciais distintas para o tipo `Integer`. Nota-se que a disposição prévia dos dados dita drasticamente o desempenho das estruturas elementares: enquanto o cenário **Ordenado** representa o melhor caso para algoritmos adaptativos (como *InsertionSort* e *BubbleSort*), reduzindo o custo computacional para $O(n)$, o cenário **Invertido** impõe a latência máxima ao forçar o pior caso de movimentações na memória ($O(n^2)$).

Por outro lado, o cenário **Aleatório** reflete o caso médio de execução esperado. Já as entradas com elementos **Repetidos** demonstram latências inferiores ao caso aleatório, favorecendo especialmente algoritmos mais sofisticados (como o *QuickSort*), que tiram proveito de otimizações de particionamento e da drástica redução no número de chaves únicas a serem movimentadas.

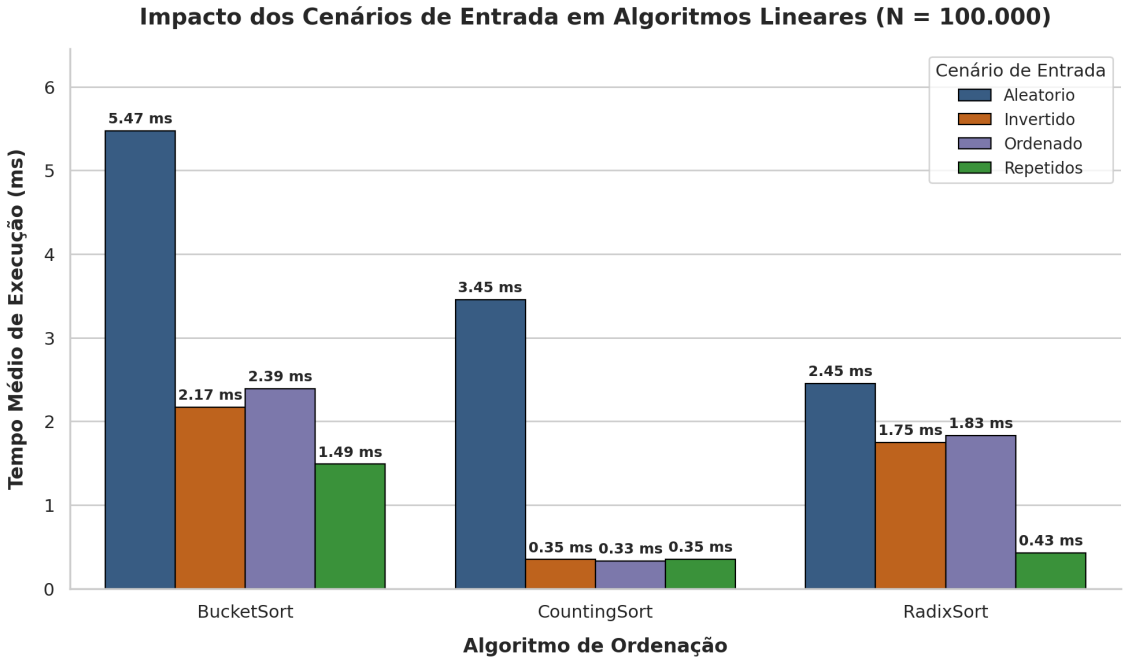


Figura 8. Impacto dos cenários de entrada nos algoritmos de complexidade linear ($N = 100.000$, tipo primitivo `int[]`).

A Figura 8 ilustra o tempo médio de execução dos três algoritmos de tempo linear (*CountingSort*, *RadixSort* e *BucketSort*) operando sobre `int[]` para $N = 100.000$ sob as quatro configurações iniciais. Embora a complexidade assintótica $O(n + k)$ mantenha todos os tempos abaixo de 6 ms, observa-se que o cenário **Aleatório** impõe a maior latência (ex: $3{,}45$ ms no *CountingSort* contra $\approx 0{,}35$ ms nos demais cenários). Isso ocorre porque acessos aleatórios ao vetor de frequências geram falhas de localidade espacial na memória cache (*cache misses*), enquanto sequências ordenadas, invertidas ou de baixa amplitude (**Repetidos**) maximizam o reaproveitamento de linhas de cache da CPU.

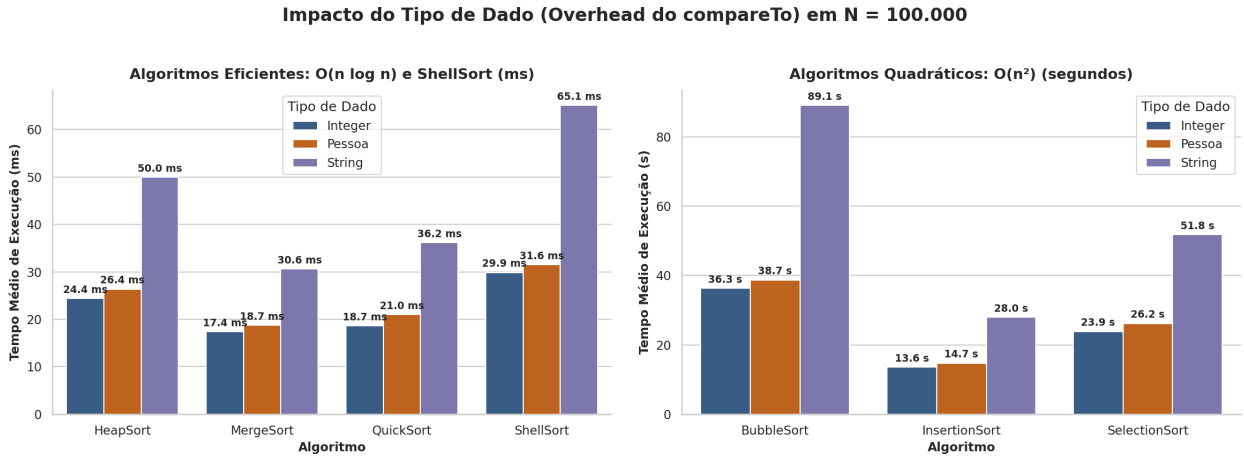


Figura 9. Análise do impacto e sobrecarga de abstração entre diferentes tipos de dados (`Integer`, `Pessoa` e cadeias de caracteres `String`).

Ao comparar a Tabela 2 e a Figura 9, nota-se que a classe `Pessoa` gerou um *overhead* desprezível (o *QuickSort* passou de $18{,}67$ ms no `Integer` para $20{,}97$ ms em `Pessoa`). Isso evidencia que, em Java, os objetos complexos são ordenados apenas através da troca de suas referências de 64-bits na *Heap*, sem a necessidade de transitar a "carga útil" de seus atributos na memória. Em compensação, a ordenação de objetos `String` elevou sensivelmente a latência, fato que será aprofundado na Seção 4.6.

4.4. Análise de Dispersão e Variabilidade Estatística

A partir dos dados consolidados na Tabela 2 e nas estatísticas globais coletadas, a análise das métricas de centralidade e dispersão (Média μ, Desvio Padrão σ, Mediana Q_2 e Amplitude) revela comportamentos fundamentais sobre a estabilidade dos algoritmos e a dinâmica de execução na JVM:

1. **Determinismo dos Algoritmos Quadráticos (CV $< 0{,}2\%$):** Os algoritmos de classe $O(n^2)$ (*SelectionSort*, *InsertionSort* e *BubbleSort*) registraram Coeficientes de Variação ($\text{CV} = \frac{\sigma}{\mu} \times 100$) inferiores a $0{,}2\%$ para $N = 100.000$. Como observado na Tabela 2, o reduzido desvio padrão ($\sigma \approx \pm 23{,}84$ ms) frente a um tempo médio elevado ($\mu \approx 23.870{,}97$ ms no *SelectionSort*) evidencia o caráter determinístico do laço quadrático. Diante do volume de $\approx 5 \times 10^9$ operações de comparação, as micro-oscilações do sistema operacional tornam-se estatisticamente irrelevantes.
2. **Sensibilidade dos Algoritmos de Alocação Dinâmica ($O(n)$):** Em contrapartida, os algoritmos não-comparativos (*BucketSort*, *RadixSort* e *CountingSort*) apresentaram variações percentuais significativamente mais expressivas. No cenário Aleatório (Tabela 2), o CV variou de $1{,}31\%$ (*RadixSort* em $N = 100.000$) a $11{,}44\%$ (*BucketSort* em $N = 100.000$) e atingiu $17{,}60\%$ no *RadixSort* para $N = 1.000.000$ (chegando a picos de $\approx 28{,}9\%$ no *BucketSort* nos demais cenários da base experimental). Como seus tempos médios situam-se em patamares reduzidos ($\mu \approx 2$ ms a 5 ms em $N = 100.000$), flutuações na alocação dinâmica de memória *Heap* (criação de baldes e vetores auxiliares de contagem) e atuações pontuais do *Garbage Collector* (GC) exercem impacto proporcionalmente maior sobre o desvio padrão.

3. **Assimetria de Média e Mediana frente ao Efeito JIT/Warm-up:** Para algoritmos de alta eficiência (*QuickSort* e *MergeSort*), nota-se uma leve assimetria positiva ($\mu > Q_2$). Nas medições para $N = 100.000$ no *QuickSort* com entradas de `String` repetidas, por exemplo, a mediana estabeleceu-se em $6{,}02$ ms e o valor mínimo em $5{,}99$ ms, enquanto o tempo máximo atingiu $10{,}93$ ms, elevando a média para $6{,}60$ ms. Esse pico isolado decorre da compilação *Just-In-Time* (JIT) e otimizações tardias da máquina virtual, confirmando que a **mediana reflete de forma mais precisa o desempenho do algoritmo em regime permanente**.

4. **Validação do Protocolo Experimental:** A reduzida amplitude observada entre os valores de Mínimo e Máximo na maioria das configurações confirma a adequação da metodologia utilizada. As 3 rodadas de pré-aquecimento (*warm-up*) e o isolamento do ambiente de teste foram eficazes em mitigar ruídos externos, assegurando a reprodutibilidade dos dados compilados na Tabela 2.

4.5. Análise de Perfil Individual de Desempenho

Para compreender a estabilidade interna, é produtivo representar algumas classes de algoritmos e observar sua curva em diferentes configurações de entrada simultaneamente.

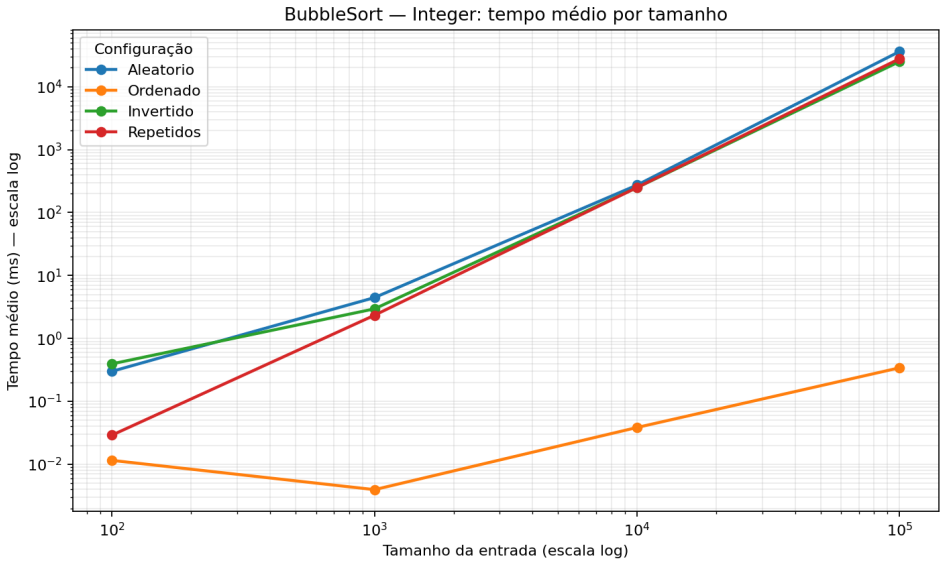


Figura 10. Variação de tempo do *Bubble Sort* perante todas as configurações de entrada.

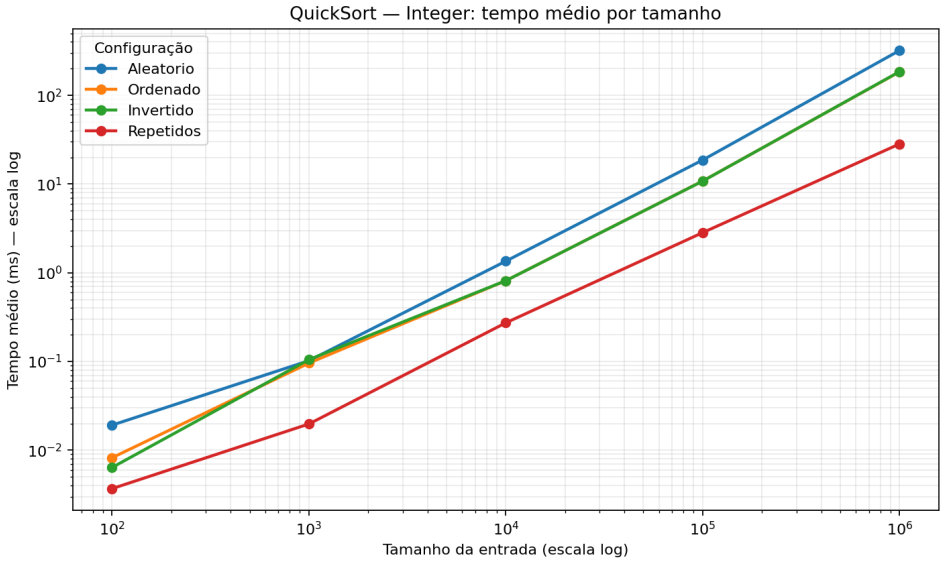


Figura 11. Variação de tempo do *Quick Sort* perante todas as configurações de entrada.

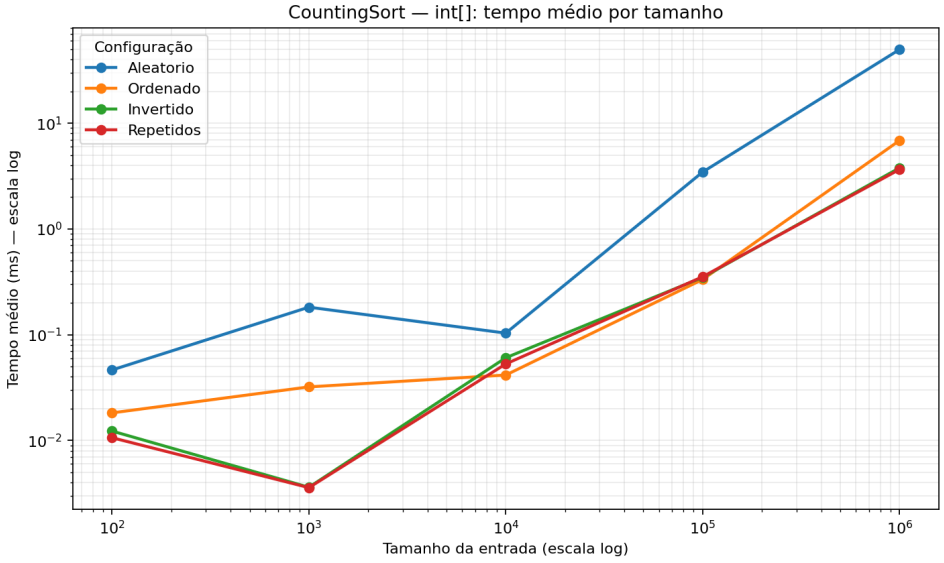


Figura 12. Variação de tempo do *Counting Sort* perante todas as configurações de entrada.

As Figuras 10 e 11 traçam o perfil do *BubbleSort* e do *QuickSort* sob os quatro cenários testados. No perfil individual do *BubbleSort*, observa-se a separação das linhas: enquanto a curva representativa da entrada "Ordenado" mantém-se na base do gráfico (melhor caso $O(n)$), as linhas de "Aleatório" e "Invertido" apresentam crescimento acentuado, compatível com o comportamento quadrático $O(n^2)$. Para o *QuickSort*, nota-se um agrupamento coeso entre os cenários ordenado, invertido e aleatório, demonstrando a eficácia da escolha do pivô central em evitar a degradação para $O(n^2)$, enquanto o cenário "Repetidos" apresenta queda expressiva de tempo graças ao particionamento em 3 vias.

Por fim, a Figura 12 ilustra o comportamento do *CountingSort* operando sobre tipos primitivos. Diferente dos algoritmos baseados em comparação, as suas curvas caminham aglomeradas na base do gráfico, evidenciando a sua eficiência linear $O(n + k)$ independentemente do estado inicial dos dados.

É interessante notar, contudo, uma anomalia (uma "queda" no tempo de execução) nas transições dos tamanhos iniciais ($N \leq 10^4$). Apesar da aplicação metodológica de 3 rodadas de *warm-up*, as execuções nestas ordens de grandeza ocorrem em frações de milissegundo. Nesses volumes reduzidos, o número total de iterações nos laços do algoritmo não é suficiente para atingir os limiares de compilação avançada (*Tier 4 / C2 Compiler*) do motor JIT da JVM. Apenas quando o tamanho da entrada cresce e o limiar de gatilho do JIT é alcançado, o código é otimizado em profundidade, provocando uma queda na curva de latência antes de retomar o seu crescimento linear natural.

Nota: Por restrições de formatação e concisão deste relatório, apresentamos apenas os perfis emblemáticos do Bubble Sort, Quick Sort e Counting Sort para representar algumas das complexidades. Os gráficos individuais completos para todos os dez algoritmos, sob todas as permutações e cenários, encontram-se disponíveis no repositório do projeto no GitHub e nos Apêndices suplementares.

4.6. Análise de Anomalias e Padrões Não-Óbvios

A análise conjunta das métricas da Tabela 2 e dos perfis individuais destaca padrões práticos relevantes observados nos testes:

- **Comportamento do *SelectionSort* nas Configurações Testadas:** Diferente de métodos adaptativos, o *SelectionSort* apresentou tempos de execução muito próximos entre si nas diferentes configurações avaliadas ($19{,}3$ s a $24{,}0$ s para $N = 100.000$ em `Integer`), o que corrobora o comportamento esperado da execução de $\frac{n(n-1)}{2}$ comparações independentemente da ordem inicial dos dados.
- **Desempenho do *InsertionSort* em Baixa Entropia:** Nos cenários pré-ordenados para $N = 100.000$ em `Integer`, o *InsertionSort* registrou tempos expressivamente baixos ($\approx 0{,}43$ ms), superando algoritmos de complexidade assintótica superior como o *MergeSort* ($\approx 8{,}78$ ms) e o *QuickSort* ($\approx 10{,}82$ ms) nessas condições específicas de entrada ordenada.
- **Eficiência do *ShellSort* em Entradas Invertidas e Ordenadas:** Nos testes com `Integer` em $N = 100.000$, o *ShellSort* apresentou latências inferiores às de algoritmos clássicos como *QuickSort* e *MergeSort* nos cenários ordenado ($\approx 4{,}40$ ms) e invertido ($\approx 7{,}24$ ms), indicando que os saltos decrescentes mitigaram de forma eficiente a movimentação de elementos distantes.
- **Variação de Desempenho no *QuickSort* com Chaves Repetidas:** A introdução de alta densidade de chaves duplicadas no intervalo $[0, 10)$ reduziu o tempo médio do *QuickSort* em `Integer` para $N = 100.000$ de $18{,}67$ ms (caso aleatório) para $2{,}83$ ms (caso repetido), resultado associado ao particionamento de 3 vias implementado, que agrupa chaves iguais de forma eficiente.
- **Impacto da Complexidade de Comparação em Objetos `String`:** Conforme a Tabela 2 ($N = 100.000$), a substituição de tipos `Integer` por objetos `String` elevou a latência do *QuickSort* em $\approx 93{,}6\%$ (de $18{,}67$ ms para $36{,}15$ ms), enquanto

o *BubbleSort* apresentou um incremento de $\approx 145{,}4\%$ (de $36.289{,}66$ ms para $89.069{,}35$ ms). Esse comportamento sugere que o custo adicional da chamada ao método `compareTo()` em cadeias de caracteres atua de forma proporcionalmente mais severa sobre algoritmos com maior volume total de comparações.

5. Conclusões

Os experimentos conduzidos neste estudo permitiram observar comportamentos compatíveis com a análise de complexidade assintótica, evidenciando como o tamanho e as características das entradas influenciam o desempenho dos algoritmos de ordenação. Os resultados indicaram diferenças expressivas entre os métodos de complexidade quadrática, como Bubble Sort, Selection Sort e Insertion Sort, e os algoritmos de crescimento assintótico inferior, especialmente à medida que o volume de dados aumenta. Nas configurações testadas com vetores de 100.000 elementos ou mais, os algoritmos quadráticos apresentaram tempos de execução elevados, reforçando as limitações práticas dessas abordagens para grandes entradas, embora seu desempenho também dependa das características dos dados e da implementação.

Os resultados também indicaram que Counting Sort e Radix Sort podem apresentar desempenho competitivo na ordenação de inteiros, aproveitando características específicas dos valores de entrada. Entretanto, sua eficiência depende de parâmetros como a amplitude numérica e a quantidade de dígitos, não sendo possível generalizar sua superioridade para todos os cenários. Além disso, fatores como a disposição inicial dos elementos, o custo das operações de comparação e as características das implementações influenciaram os tempos observados, demonstrando que a complexidade assintótica, embora fundamental para compreender o crescimento do custo computacional, não explica isoladamente todas as diferenças de desempenho.

Em aplicações práticas de desenvolvimento de software, a escolha do algoritmo deve considerar o tamanho dos dados, suas características, os requisitos de estabilidade, o consumo de memória e as particularidades da implementação. Quick Sort, Merge Sort e Heap Sort apresentam propriedades distintas que podem torná-los adequados a diferentes situações. Da mesma forma, algoritmos não comparativos podem ser vantajosos quando os dados atendem às condições necessárias para sua aplicação. Assim, os resultados deste estudo reforçam a importância de combinar a análise teórica com avaliações experimentais para orientar a escolha de métodos de ordenação, considerando as condições específicas de cada problema.

Disponibilidade de Repositório e Dados

O código-fonte integral das implementações, as suítes de testes de corretude, o motor automatizado de *benchmarking* e a base de dados brutos coletados neste experimento encontram-se disponíveis em repositório público para auditoria e reprodutibilidade através do link: `https://github.com/Beforg/acd-tp1`.

Declaração de Uso de Inteligência Artificial

Em conformidade com as diretrizes de integridade acadêmica e transparência na pesquisa, declara-se que ferramentas de Inteligência Artificial Generativa foram utilizadas como recurso auxiliar durante o desenvolvimento deste trabalho, restritas às seguintes atividades:

1. **Apoio e Aprofundamento Teórico:** Consulta e síntese de conceitos fundamentais de análise assintótica e estruturas de dados, tomando como referência primária a literatura clássica de algoritmos [Cormen et al., 2012, Goodrich e Tamassia, 2013].
2. **Revisão e Verificação de Código:** Inspeção estática das implementações dos dez algoritmos de ordenação em Java para auxílio na identificação de gargalos de desempenho, otimização de pilha no *QuickSort* e garantia da corretude lógica.
3. **Automação de Testes Unitários:** Auxílio na geração da classe de verificação de corretude (`TestesCorretude`), utilizada para validar o comportamento dos algoritmos sob diferentes cenários (vetores vazios, unitários, duplicados e ordenados).
4. **Revisão e Formatação do Relatório Técnico:** Suporte no refinamento estilístico, coesão textual, correção gramatical e estruturação do documento no padrão da Sociedade Brasileira de Computação (SBC).

Toda a concepção do experimento, execução dos benchmarks, coleta e validação dos dados estatísticos e interpretação dos resultados foram realizadas inteiramente pelos autores.

Referências

Cormen, T. H., Leiserson, C. E., Rivest, R. L., e Stein, C. (2012). *Algoritmos: Teoria e Prática*. Elsevier, 3ª edição.

Goodrich, M. T. e Tamassia, R. (2013). *Estruturas de Dados e Algoritmos em Java*. Bookman, 5ª edição.